Báo cáo quá trình học việc

Nền tảng hạ tầng: Linux doanh nghiệp, Ảo hóa và Container

Người trình bày: Lê Tấn Lực (Intern) | Tháng 11/2026

Mở đầu

Trong thời gian học việc vừa qua, em tập trung tìm hiểu kiến thức nền tảng về hạ tầng hệ thống, gồm ba mảng chính: hệ điều hành Linux dòng doanh nghiệp, ảo hóa và container. Em có cài **Red Hat Enterprise Linux (RHEL) 10.2** và Docker để làm quen. Tuy nhiên, thời gian thực hành thực tế của em còn ít, nên báo cáo này tập trung trình bày **phần lý thuyết em đã hiểu**. Các câu lệnh đi kèm chỉ là ví dụ minh họa cho lý thuyết, không phải bài lab em đã làm hoàn chỉnh.

Em chọn tìm hiểu theo RHEL vì đây là hệ điều hành được nhiều doanh nghiệp sử dụng, và cũng là nền tảng của các chứng chỉ Red Hat em định hướng sau này. RHEL 10 ra mắt tháng 5/2025. Bản 10.2 là bản cập nhật nhỏ (minor release), phát hành tháng 5/2026. Red Hat ra bản minor khoảng 6 tháng một lần, và các bản trong cùng dòng 10 giữ tương thích với nhau. Khác với các bản Linux cộng đồng, máy RHEL phải **đăng ký subscription** thì mới truy cập được kho phần mềm. Kho chính gồm hai repo: **BaseOS** chứa phần lõi hệ điều hành, **AppStream** chứa ứng dụng, ngôn ngữ lập trình và công cụ. Với mục đích học tập cá nhân, Red Hat có gói **Developer Subscription for Individuals**, miễn phí và dùng được tối đa 16 hệ thống.

Thay vì học từng chủ đề rời rạc, em sắp xếp kiến thức theo đúng thứ tự mà một máy chủ cần có để đi vào hoạt động. Trước hết phải có máy, và trong doanh nghiệp đó thường là máy ảo chạy trên hypervisor. Máy cần được nối mạng để giao tiếp với các máy khác, cần ổ đĩa để lưu dữ liệu, cần tài khoản người dùng với quyền hạn phù hợp, cần chạy dịch vụ và được firewall bảo vệ. Khi hệ thống đã chạy thì phải theo dõi được tài nguyên CPU, RAM, disk. Cuối cùng là container, cách đóng gói và vận hành ứng dụng phổ biến hiện nay. Sắp xếp như vậy giúp em thấy phần trước luôn là tiền đề cho phần sau.

Mỗi phần dưới đây gồm khái niệm, cách hoạt động, những điểm riêng của RHEL 10 cần lưu ý, và một vài ví dụ minh họa.

1. Hypervisor và vòng đời VM

Đây là "nền móng" của mọi phần phía sau: trước khi có một máy chủ RHEL để cấu hình, ta cần có máy, và trong doanh nghiệp, máy đó hầu hết là máy ảo (VM).

Hypervisor là gì

Hypervisor là lớp phần mềm chia tài nguyên của một máy vật lý (CPU, RAM, ổ đĩa, card mạng) thành nhiều máy ảo độc lập. Mỗi VM "tưởng" rằng mình đang chạy trên một máy thật riêng, có CPU ảo (vCPU), RAM, ổ đĩa ảo và card mạng ảo. Thực tế, hypervisor đứng giữa điều phối để các VM dùng chung phần cứng mà không ảnh hưởng lẫn nhau. CPU hiện đại có tính năng hỗ trợ ảo hóa (Intel VT-x, AMD-V), nhờ đó VM đạt hiệu năng gần với máy thật.

Ví dụ: một máy chủ vật lý 32 CPU, 128 GB RAM nếu chỉ chạy một ứng dụng thì phần lớn tài nguyên bị bỏ phí. Khi ảo hóa, cùng máy đó có thể chạy 10 đến 15 VM cho web, database, mail. Doanh nghiệp tiết kiệm được phần cứng, điện, chỗ đặt máy, và tạo máy mới chỉ mất vài phút thay vì chờ mua thiết bị.

Hai loại hypervisor

Type 1 (bare-metal) chạy trực tiếp trên phần cứng, ví dụ VMware ESXi, Microsoft Hyper-V, KVM. Loại này dùng trong datacenter và cloud vì hiệu năng cao và ổn định. **Type 2 (hosted)** chạy như một ứng dụng trên hệ điều hành có sẵn, ví dụ VirtualBox, VMware Workstation. Loại này phù hợp để cá nhân học tập, thử nghiệm.

Có hai điểm em thấy thú vị. Thứ nhất, Hyper-V trông giống một tính năng của Windows nhưng thực chất là Type 1: khi bật Hyper-V, hypervisor được nạp xuống dưới cùng, còn Windows trở thành một phân vùng chạy trên nó. Thứ hai, bản thân RHEL cũng có thể làm hypervisor. **KVM** nằm sẵn trong kernel Linux, kết hợp với QEMU để giả lập phần cứng và **libvirt** để quản lý. Hypervisor Nitro dưới máy ảo EC2 của AWS cũng dựa trên KVM.

Một VM gồm những gì

Về bản chất, một VM là một nhóm file: một file cấu hình mô tả phần cứng ảo (số vCPU, dung lượng RAM, card mạng nối vào đâu), và một hoặc nhiều file ổ đĩa ảo chứa toàn bộ hệ điều hành và dữ liệu. Mỗi hypervisor có định dạng ổ đĩa riêng: VDI (VirtualBox), VHDX (Hyper-V), VMDK (VMware), QCOW2 (KVM). Ổ đĩa ảo có hai kiểu cấp phát. **Thin** (cấp phát động): file lớn dần theo dữ liệu thật, tiết kiệm chỗ. **Thick** (cấp phát cố định): chiếm đủ dung lượng ngay từ đầu, hiệu năng ổn định hơn. Vì VM chỉ là file nên có thể sao chép, di chuyển, sao lưu dễ dàng.

Hypervisor còn cho phép **overcommit**: tổng vCPU hoặc RAM cấp cho các VM được phép vượt tài nguyên thật, vì không phải VM nào cũng dùng hết cùng lúc. Overcommit giúp tận dụng tài nguyên, nhưng nếu quá tay thì khi các VM cùng bận, tất cả sẽ chậm.

Riêng với RHEL 10, cần lưu ý yêu cầu CPU hỗ trợ tập lệnh **x86-64-v3** (có AVX2, từ CPU Intel đời Haswell và AMD đời Excavator trở đi). Với máy ảo, CPU thật hỗ trợ là chưa đủ: hypervisor còn phải "chuyển" các tính năng này vào VM. Nếu hypervisor ẩn bớt tính năng CPU, RHEL 10 sẽ không khởi động được và báo lỗi glibc rằng CPU không được hỗ trợ. Đây là ví dụ cho thấy cấu hình hypervisor ảnh hưởng trực tiếp đến hệ điều hành bên trong.

Vòng đời của một VM

Một VM thường đi qua năm giai đoạn. **Tạo**: chọn loại hệ điều hành, cấp vCPU, RAM, ổ đĩa, card mạng. **Cài đặt và cấu hình**: cài hệ điều hành, đặt IP, tạo user, cài dịch vụ (chính là nội dung các phần 2 đến 5). **Vận hành**: theo dõi tài nguyên, điều chỉnh khi thiếu, như tăng RAM hay thêm ổ. **Bảo trì**: snapshot trước khi cập nhật, backup định kỳ, clone ra máy mới. **Thu hồi**: khi hết sử dụng thì backup lần cuối nếu cần rồi xóa để giải phóng tài nguyên. Với RHEL, trước khi xóa nên hủy đăng ký subscription để trả lại suất sử dụng.

Trong doanh nghiệp, người ta thường không cài từng VM từ đầu mà tạo một **template** (máy mẫu đã cài sẵn hệ điều hành và cấu hình chuẩn), rồi **clone** ra hàng loạt máy giống nhau. Khi clone máy Linux cần làm "sạch" các thông tin phải là duy nhất như machine-id, SSH host key, hostname. Hypervisor Type 1 còn hỗ trợ **live migration**: chuyển một VM đang chạy sang máy vật lý khác mà không phải tắt, rất hữu ích khi cần bảo trì phần cứng.

Snapshot khác backup ở điểm nào

Snapshot lưu lại trạng thái VM tại một thời điểm. Sau khi chụp, ổ đĩa gốc được "đóng băng", mọi thay đổi mới được ghi vào một file chênh lệch riêng. Vì vậy, muốn quay về chỉ cần bỏ file chênh lệch, gần như tức thì. Nhưng chính cơ chế này cho thấy snapshot **phụ thuộc vào ổ đĩa gốc** và nằm ngay cạnh VM: nếu ổ cứng máy chủ hỏng thì mất cả VM lẫn snapshot. **Backup** là bản sao đầy đủ, độc lập, lưu ở nơi khác, nên khôi phục được kể cả khi mất toàn bộ máy chủ. Có thể ví snapshot như nút "Undo" trong Word, còn backup như bản sao tài liệu cất trong két sắt ở một nơi

khác. Nguyên tắc **3-2-1**: giữ 3 bản dữ liệu, trên 2 loại thiết bị lưu trữ khác nhau, trong đó 1 bản đặt ở nơi khác.

Ví dụ tình huống: trước khi chạy dnf update để nâng cấp một máy chủ RHEL, quản trị viên chụp snapshot. Nếu bản cập nhật gây lỗi, chỉ mất vài giây để quay lại. Nếu cập nhật thành công thì nên xóa snapshot, vì giữ lâu sẽ khiến file chênh lệch phình to và VM chạy chậm dần. Backup thì vẫn chạy hằng đêm, độc lập với snapshot. Khi đã có máy, câu hỏi tiếp theo là làm sao để các máy kết nối được với nhau và ra Internet.

2. Network: IP, routing, DNS và kết nối giữa các VM

Mỗi máy trong mạng cần một **địa chỉ IP** và biết mình thuộc mạng nào nhờ **subnet mask**, thường viết gọn theo kiểu CIDR. Ví dụ **192.168.56.10/24**: 24 bit đầu là phần mạng (192.168.56), 8 bit cuối là phần máy, nên mạng có 256 địa chỉ. Trong đó .0 là địa chỉ mạng, .255 là địa chỉ broadcast, còn 254 địa chỉ dùng được cho máy. Mạng nội bộ dùng ba dải IP riêng (private), các dải này không đi thẳng ra Internet được: 10.0.0.0/8, 172.16.0.0/12 và 192.168.0.0/16.

Routing và gateway

Khi gửi gói tin, máy so IP đích với mạng của mình. Nếu đích cùng mạng, máy gửi thẳng tới, dùng ARP để tìm địa chỉ MAC của máy đích. Nếu đích ở mạng khác, gói tin được gửi cho **gateway** (router) chuyển tiếp. Quy tắc chọn đường nằm trong **bảng định tuyến**: máy chọn route cụ thể nhất (prefix dài nhất) khớp với đích. Nếu không route nào khớp thì gói tin đi theo **default route**. Khi có nhiều route cùng đích, route có **metric** nhỏ hơn được ưu tiên. Ví dụ bảng định tuyến của một máy có hai card mạng:

```
$ ip route
default via 10.0.2.2 dev enp0s3 proto dhcp metric 100
192.168.56.0/24 dev enp0s8 proto kernel scope link src 192.168.56.10 metric 101
```

Cách đọc: gói tin đến mạng 192.168.56.0/24 đi thẳng qua card enp0s8, không cần gateway. Mọi đích khác, kể cả Internet, đi theo default route qua gateway 10.0.2.2 trên card enp0s3. Vì vậy, thông thường chỉ card dùng để ra Internet mới khai báo gateway, tránh để máy có hai default route gây nhầm đường.

Các chế độ mạng của VM

Hypervisor cho VM nhiều chế độ mạng, mỗi chế độ trả lời một câu hỏi khác nhau. **NAT**: VM ra được Internet nhưng bên ngoài không truy cập vào được. **Host-only**: máy thật và các VM thấy nhau nhưng không ra Internet. **Bridged**: VM giống như một máy thật trong mạng LAN, nhận IP cùng dải với các máy khác. **Internal**: chỉ các VM nói chuyện với nhau. Trên Hyper-V, các khái niệm tương ứng là External, Internal và Private switch. Một cách bố trí phổ biến là cho mỗi VM hai card mạng: một card NAT để tải phần mềm, một card mạng nội bộ để quản trị và để các máy kết nối với nhau.

Quản lý mạng trên RHEL 10

Trên RHEL 10, **NetworkManager** là công cụ quản lý mạng. Mỗi cấu hình là một **connection profile** gắn với một card mạng (device). Điểm thay đổi quan trọng: RHEL 10 đã **bỏ hẳn định dạng ifcfg** cũ trong /etc/sysconfig/network-scripts/. Profile giờ được lưu dạng **keyfile** (đuôi .nmconnection) trong /etc/NetworkManager/system-connections/. Vì vậy các tài liệu cũ hướng dẫn sửa file ifcfg-eth0 không còn áp dụng được. Có thể cấu hình bằng dòng lệnh nmcli, giao diện văn bản nmtui, hoặc giao diện web Cockpit. Ví dụ đặt IP tĩnh và DNS cho một card mạng:

```
nmcli con mod enp0s8 ipv4.method manual ipv4.addresses 192.168.56.10/24
```

```
nmcli con mod enp0s8 ipv4.dns "8.8.8.8 1.1.1.1"
nmcli con up enp0s8
```

DNS và phân giải tên

Con người nhớ tên, máy tính dùng IP, nên DNS làm nhiệm vụ chuyển tên thành IP. Linux tra tên theo thứ tự khai báo trong /etc/nsswitch.conf: thường là file /etc/hosts trước, DNS sau. File /etc/resolv.conf chứa địa chỉ DNS server nhưng do NetworkManager tự sinh ra, nên khai báo DNS phải qua NetworkManager, sửa tay sẽ bị ghi đè. Một số loại bản ghi DNS cơ bản: **A** (tên ra IPv4), **AAAA** (tên ra IPv6), **CNAME** (bí danh), **PTR** (IP ra tên), **MX** (máy chủ mail).

Điều quan trọng nhất em rút ra ở phần này là **thứ tự kiểm tra khi mất kết nối**, đi từ lớp dưới lên: card mạng đã có IP chưa (ip addr), có đường đi chưa (ip route), ping được gateway không, ping được IP đích không, rồi mới đến phân giải tên (DNS) và kiểm tra cổng dịch vụ (ss, curl). Ví dụ: nếu ping 8.8.8.8 được mà ping google.com không được, lỗi nằm ở DNS chứ không phải đường mạng. Khi mạng đã thông, việc tiếp theo là chuẩn bị chỗ lưu dữ liệu cho máy chủ.

3. Storage: partition, LVM, filesystem, mount

Một nguyên tắc cơ bản là không để dữ liệu ứng dụng chung phân vùng với hệ điều hành. Nếu log hoặc dữ liệu làm đầy ổ, cả máy chủ có thể ngừng hoạt động. Để lưu trữ dữ liệu trên Linux cần đi qua các lớp: **ổ đĩa** → **partition** → (tùy chọn) **LVM** → **filesystem**, và cuối cùng là **mount** vào một thư mục để sử dụng.

Ổ đĩa và partition

Linux coi mỗi ổ đĩa là một **block device** trong /dev. Tên thiết bị phụ thuộc loại ổ: /dev/sda, /dev/sdb (SATA/SCSI), /dev/nvme0n1 (NVMe), /dev/vda (ổ ảo virtio trên KVM). Partition chia ổ thành các vùng độc lập theo một trong hai chuẩn bảng phân vùng. **MBR** là chuẩn cũ: tối đa ổ 2 TB và 4 partition chính. **GPT** là chuẩn mới: hỗ trợ ổ rất lớn, mặc định tới 128 partition, đi kèm UEFI. Công cụ chia ổ phổ biến là fdisk, gdisk và parted.

LVM: Physical Volume, Volume Group, Logical Volume

LVM gồm ba lớp. **Physical Volume (PV)** là các ổ đĩa hoặc partition thật. **Volume Group (VG)** là một "bể chứa" gom dung lượng của nhiều PV lại, được chia thành các đơn vị nhỏ gọi là extent (mặc định 4 MiB). **Logical Volume (LV)** là phần cắt ra từ bể đó để dùng như một ổ đĩa. Lợi ích lớn nhất là linh hoạt: khi hết chỗ, chỉ cần thêm ổ vào VG rồi nới rộng LV, không phải chia lại ổ từ đầu. Khi cài RHEL với chế độ phân vùng tự động, hệ điều hành mặc định đã nằm trên LVM, trong VG tên "rhel" với các LV root và swap. Từ RHEL 9 trở đi còn có **LVM devices file** (/etc/lvm/devices/system.devices): LVM chỉ làm việc với các thiết bị có tên trong file này, tránh "nhận nhầm" ổ đĩa của hệ thống khác.

Filesystem và mount

XFS là filesystem mặc định của RHEL 10, mạnh với file lớn và ghi song song, nhưng **chỉ mở rộng được, không thu nhỏ được**. RHEL vẫn hỗ trợ **ext4** (có thể thu nhỏ khi đã umount), còn Btrfs không được hỗ trợ. Lệnh mount gắn filesystem vào một thư mục nhưng chỉ có hiệu lực đến lần khởi động lại. Muốn mount tự động thì khai báo trong **/etc/fstab**, mỗi dòng 6 trường: thiết bị, điểm mount, loại filesystem, tùy chọn, dump, thứ tự fsck. Nên dùng **UUID** thay cho tên thiết bị, vì tên kiểu /dev/sdb có thể thay đổi khi thêm bớt ổ. Một dòng sai trong fstab có thể khiến máy vào emergency mode khi khởi động, nên cần kiểm tra bằng mount -a trước khi reboot. Ví dụ quy trình tạo vùng lưu trữ /data từ một ổ mới:

```
pvcreate /dev/sdb1                    # khởi tạo PV
```

```
vgcreate vg_data /dev/sdb1              # tạo bể chứa VG
lvcreate -n lv_web -L 5G vg_data        # cắt 5 GB thành LV
mkfs.xfs /dev/vg_data/lv_web            # tạo filesystem XFS
# /etc/fstab:  UUID=<uuid>  /data  xfs  defaults  0 0
lvextend -r -L +3G /dev/vg_data/lv_web  # sau này cần thêm: nới LV và filesystem
```

Ngoài ra, **swap** là vùng đĩa dùng làm "RAM dự phòng" khi RAM thật đầy. Swap giúp máy không bị treo ngay khi thiếu RAM, nhưng chậm hơn RAM rất nhiều. Có chỗ chứa dữ liệu rồi, câu hỏi tiếp theo là ai được phép đọc, ghi vào đó.

4. User, group và permission

Linux là hệ điều hành nhiều người dùng. Mỗi tài khoản có một mã số **UID**: **root** có UID 0 và toàn quyền, tài khoản hệ thống dành cho dịch vụ (như apache) có UID dưới 1000, người dùng thường bắt đầu từ 1000. Thông tin được lưu trong ba file. **/etc/passwd**: tên, UID, GID, thư mục home, shell. **/etc/shadow**: mật khẩu đã băm và chính sách hết hạn, chỉ root đọc được. **/etc/group**: danh sách nhóm và thành viên.

Mỗi user có một **nhóm chính** (primary group). Trên RHEL, khi tạo user, hệ thống tự tạo một nhóm trùng tên làm nhóm chính. Ngoài ra user có thể thuộc nhiều **nhóm phụ** (supplementary group) để được chia sẻ quyền. Một điểm của RHEL 10: trình cài đặt **mặc định khóa tài khoản root** và khuyến khích tạo user quản trị thuộc nhóm **wheel**, làm việc qua sudo. Cách này an toàn hơn đăng nhập thẳng bằng root.

Quyền truy cập file

Mọi file đều trả lời ba câu hỏi: ai sở hữu (owner, u), thuộc nhóm nào (group, g), và những người còn lại (other, o) được làm gì. Mỗi đối tượng có ba quyền: **r** (đọc, giá trị 4), **w** (ghi, 2), **x** (thực thi, 1). Với thư mục, ý nghĩa hơi khác: r là xem danh sách file, w là tạo hoặc xóa file bên trong, x là được đi vào thư mục. Cộng các giá trị sẽ ra con số quen thuộc, ví dụ 750 nghĩa là owner rwx, group r-x, other không có quyền gì. Quyền mặc định của file mới do **umask** quyết định, ví dụ umask 022 cho ra file 644 và thư mục 755.

Ngoài ra có ba quyền đặc biệt. **setuid** (4): chương trình chạy với quyền của chủ file, ví dụ lệnh passwd cần ghi vào /etc/shadow. **setgid** (2) trên thư mục: file mới tạo tự thuộc nhóm của thư mục. **sticky bit** (1): chỉ chủ file mới xóa được file của mình, ví dụ /tmp có quyền 1777. Khi cần phân quyền chi tiết hơn mô hình u/g/o, có thể dùng **ACL** (setfacl, getfacl).

Ví dụ tình huống: team web gồm hai bạn An và Bình cần cùng làm việc trong thư mục /data/web, người ngoài nhóm không được xem. Cách làm là tạo nhóm webteam, cho hai bạn vào nhóm, rồi đặt quyền 2770 cho thư mục. Số 2 (setgid) giúp mọi file mới đều thuộc nhóm webteam, nên hai bạn sửa được file của nhau:

```
groupadd webteam
usermod -aG webteam an          # -a để thêm nhóm, không ghi đè nhóm cũ
chown root:webteam /data/web
chmod 2770 /data/web            # kết quả: drwxrws---  root webteam  /data/web
```

sudo và nguyên tắc least privilege

Thành viên nhóm wheel được dùng sudo với toàn quyền. Tuy nhiên theo nguyên tắc **least privilege**, chỉ nên cấp đúng quyền cần thiết. Ví dụ team web chỉ cần khởi động lại dịch vụ web thì chỉ cấp đúng lệnh đó, bằng một dòng trong /etc/sudoers.d/: *%webteam ALL=(root) /usr/bin/systemctl restart httpd*. File sudoers nên luôn sửa bằng visudo, vì công cụ này kiểm tra cú pháp trước khi lưu, tránh lỗi làm mất quyền quản trị. Dịch vụ httpd nhắc đến ở đây chính là nội

dung của phần tiếp theo.

5. Service (systemd) và firewall

Trên RHEL, **systemd** là tiến trình đầu tiên chạy khi máy khởi động (PID 1), có nhiệm vụ khởi động và quản lý mọi dịch vụ. systemd quản lý theo đơn vị gọi là **unit**. Các loại unit hay gặp: **.service** (dịch vụ), **.socket**, **.timer** (lập lịch, thay thế cron), **.mount** (điểm mount, sinh ra từ fstab ở phần 3), và **.target** (nhóm unit, tương đương "runlevel" trước đây). Ví dụ multi-user.target là chế độ máy chủ không giao diện, graphical.target là chế độ có giao diện đồ họa.

Hai khái niệm dễ nhầm: **start** là chạy dịch vụ ngay bây giờ, **enable** là cho dịch vụ tự chạy mỗi khi khởi động máy. Muốn cả hai thì dùng enable --now. Unit file do gói phần mềm cài nằm trong /usr/lib/systemd/system/. Unit file do quản trị viên tự viết hoặc ghi đè nằm trong /etc/systemd/system/, và nơi này được ưu tiên hơn. Sau khi sửa unit file phải chạy systemctl daemon-reload. Log của mọi dịch vụ được **journald** thu thập, xem bằng journalctl. Trong unit file, dòng *Restart=on-failure* giúp systemd tự khởi động lại dịch vụ khi bị lỗi, còn *User=* cho dịch vụ chạy bằng user thường thay vì root. Ví dụ:

```
systemctl enable --now httpd       # chạy ngay và tự chạy khi khởi động
systemctl status httpd             # trạng thái và vài dòng log gần nhất
journalctl -u httpd -n 50          # 50 dòng log của dịch vụ httpd
```

Firewall với firewalld

firewalld là firewall mặc định của RHEL, dùng **nftables** trong kernel làm nền. firewalld tổ chức theo **zone**: mỗi card mạng thuộc một zone (mặc định là public), mỗi zone có danh sách service hoặc port được phép đi vào, còn lại đều bị chặn. Có hai lớp cấu hình. **Runtime** có hiệu lực ngay nhưng mất khi reload hoặc reboot. **Permanent** được lưu lại nhưng chỉ có hiệu lực sau khi reload. Vì vậy cách làm chuẩn là thêm --permanent rồi --reload:

```
firewall-cmd --permanent --add-service=http
firewall-cmd --reload
firewall-cmd --list-all
```

SELinux: lớp bảo vệ thứ ba

Ngoài quyền file (phần 4) và firewall, RHEL còn có **SELinux**, mặc định ở chế độ **enforcing**. Mỗi tiến trình và mỗi file đều mang một nhãn (context), và chính sách SELinux quy định nhãn tiến trình nào được truy cập nhãn file nào. Ví dụ tiến trình httpd (nhãn httpd_t) chỉ đọc được file mang nhãn httpd_sys_content_t. Nếu đổi thư mục web sang /data/web, trình duyệt sẽ báo lỗi 403 dù quyền rwx đã đúng, vì thư mục mới chưa có nhãn phù hợp. Cách xử lý đúng là gán lại nhãn bằng semanage fcontext và restorecon, không phải tắt SELinux. Lịch sử những lần bị SELinux chặn được ghi lại dưới dạng log AVC.

Từ đây có thể thấy: "web không vào được" có thể do dịch vụ đã dừng, firewall chặn, sai quyền file, hoặc SELinux chặn. Cần kiểm tra lần lượt từng lớp, giống cách kiểm tra mạng ở phần 2. Khi hệ thống đã chạy ổn, việc tiếp theo là theo dõi xem máy chủ có "khỏe" không.

6. Linux doanh nghiệp: quản lý tài nguyên máy chủ

Quản lý tài nguyên là trả lời bốn câu hỏi: CPU có quá tải không, RAM có đủ không, disk còn chỗ và có bị nghẽn không, tiến trình nào đang gây ra vấn đề. Khi có báo cáo "máy chủ chạy chậm", nên kiểm tra theo đúng thứ tự đó.

CPU và load average

Load average là số tiến trình trung bình đang chạy hoặc đang chờ tài nguyên (CPU hoặc I/O không thể ngắt), tính trong 1, 5 và 15 phút gần nhất. Con số này phải so với số CPU. Ví dụ load 3.95 trên máy 2 CPU nghĩa là lượng công việc gần gấp đôi khả năng xử lý, tức máy đang quá tải. Ba con số còn cho biết xu hướng: số 1 phút lớn hơn số 15 phút nghĩa là tải đang tăng. Trong lệnh top, dòng %Cpu có các chỉ số: **us** (ứng dụng), **sy** (kernel), **wa** (chờ disk), **id** (rảnh), **st** (thời gian hypervisor "lấy" cho VM khác, liên quan đến overcommit ở phần 1). Nếu wa cao thì nghẽn nằm ở disk chứ không phải CPU.

RAM và swap

```
$ free -h
               total   used   free   shared  buff/cache  available
Mem:           1.9Gi   620Mi  110Mi   12Mi     1.2Gi       1.1Gi
```

Ví dụ trên cho thấy cột free chỉ còn 110 MB nhưng máy không hề thiếu RAM. Linux tận dụng RAM trống làm cache (buff/cache) để tăng tốc đọc đĩa và sẵn sàng nhả ra khi ứng dụng cần, nên con số cần nhìn là **available** (1.1 GB). Dấu hiệu thiếu RAM thật là swap tăng liên tục, hoặc kernel phải kích hoạt **OOM killer** để "giết" tiến trình nhằm cứu hệ thống.

Disk

Có hai vấn đề khác nhau: hết dung lượng và nghẽn tốc độ. df -h cho biết phân vùng nào đầy, du -sh cho biết thư mục nào chiếm nhiều chỗ. Cần chú ý thêm **inode**: nếu có quá nhiều file nhỏ, hệ thống có thể hết inode (kiểm tra bằng df -i) và không ghi được file mới dù vẫn còn dung lượng. Tốc độ đọc ghi được theo dõi bằng iostat.

Tiến trình, tín hiệu và độ ưu tiên

Mỗi chương trình đang chạy là một tiến trình có mã **PID**, xem bằng ps hoặc top. Muốn dừng tiến trình thì gửi **tín hiệu** (signal). **SIGTERM (15)** yêu cầu dừng nhẹ nhàng, cho tiến trình kịp lưu dữ liệu và dọn dẹp, nên luôn thử trước. **SIGKILL (9)** buộc dừng ngay, chỉ dùng khi tiến trình không phản hồi. Độ ưu tiên CPU được điều chỉnh bằng giá trị **nice**, từ -20 (ưu tiên cao nhất) đến 19 (thấp nhất), mặc định là 0. Ví dụ tác vụ backup nên chạy với nice cao để không giành CPU của dịch vụ chính. User thường chỉ được giảm ưu tiên, muốn tăng ưu tiên phải có quyền root.

cgroups v2 và công cụ quản lý của RHEL

systemd đặt mỗi dịch vụ vào một **control group (cgroup)**, cơ chế của kernel dùng để theo dõi và giới hạn tài nguyên cho một nhóm tiến trình. RHEL 10 **chỉ còn hỗ trợ cgroups v2**, đã bỏ hẳn v1. Nhờ cơ chế này, có thể giới hạn tài nguyên cho từng dịch vụ, ví dụ không cho một ứng dụng dùng quá nửa CPU và 256 MB RAM:

```
systemctl set-property hello-app.service CPUQuota=50% MemoryMax=256M
```

RHEL còn có sẵn một số công cụ quản lý ở mức doanh nghiệp. **Cockpit** là giao diện web quản trị (cổng 9090) để xem CPU, RAM, disk, log và quản lý dịch vụ. **tuned** tối ưu hệ thống theo profile, ví dụ profile virtual-guest khi chạy trong VM. **sos report** thu thập thông tin hệ thống để gửi bộ phận hỗ trợ của Red Hat khi có sự cố. Cơ chế cgroups vừa nói cũng là nền tảng Docker dùng để giới hạn tài nguyên cho container, nội dung của phần cuối cùng.

7. Container: image, registry, volume, network, resource limit

Điều quan trọng nhất em hiểu được ở phần này: container không phải một "máy ảo nhỏ" mà là **tiến trình Linux được cô lập**, xây dựng từ chính các tính năng của kernel đã nhắc ở các phần trước.

Container khác VM ở điểm nào

VM ảo hóa phần cứng: mỗi VM có kernel và hệ điều hành riêng, nặng vài GB, khởi động mất cả phút. **Container ảo hóa ở mức hệ điều hành**: mọi container dùng chung kernel của máy chủ, chỉ đóng gói ứng dụng và thư viện cần thiết, nên thường chỉ nặng vài chục đến vài trăm MB và khởi động trong vài giây. Sự cô lập dựa trên hai cơ chế của kernel. **Namespaces** giúp mỗi container "thấy" một thế giới riêng: tiến trình (pid), mạng (net), cây thư mục (mnt), tên máy (uts), cùng ipc và user. **cgroups v2** (phần 6) giới hạn lượng CPU, RAM container được dùng. Có thể hình dung VM như những căn nhà riêng, mỗi nhà có móng riêng. Container giống các căn hộ chung cư: dùng chung móng (kernel) nhưng mỗi căn có cửa khóa riêng.

Đổi lại, container cô lập kém hơn VM vì dùng chung kernel: một lỗ hổng ở kernel có thể ảnh hưởng mọi container. Container Linux cũng cần kernel Linux để chạy. Vì vậy doanh nghiệp thường kết hợp cả hai: chạy container bên trong VM, giống mô hình trong sơ đồ ở phần network bên dưới.

Kiến trúc Docker

Docker gồm nhiều lớp. Lệnh **docker** (CLI) gửi yêu cầu tới **dockerd**, một dịch vụ systemd như ở phần 5. dockerd giao cho **containerd** quản lý vòng đời container, và containerd gọi **runc** để thực sự tạo namespaces và cgroups. Image và container theo chuẩn mở **OCI**, nên image build bằng Docker vẫn chạy được trên Podman hay Kubernetes. Trên RHEL, công cụ container mặc định của Red Hat là **Podman** (không cần daemon, chạy được không cần root). Docker được cài từ repo chính thức của Docker dành cho RHEL. Một lưu ý bảo mật: thành viên nhóm docker điều khiển được dockerd đang chạy bằng root, nên có quyền gần như root trên máy. Theo nguyên tắc least privilege (phần 4), chỉ thêm vào nhóm này những người thật sự cần.

Image: bản đóng gói chỉ đọc, xây theo từng lớp

Image là bản đóng gói chỉ đọc gồm ứng dụng, thư viện và cấu hình cần để chạy. Mỗi lệnh trong **Dockerfile** tạo ra một **layer**. Các layer xếp chồng lên nhau, và layer giống nhau được dùng chung giữa các image. Khi chạy, Docker thêm một **lớp ghi mỏng** trên cùng cho container, còn image bên dưới không bị sửa, nhờ đó một image chạy được nhiều container. Docker cũng cache layer: bước nào không đổi thì lần build sau dùng lại. Vì vậy nên đặt bước ít thay đổi (cài thư viện) lên trước, bước hay thay đổi (copy code) xuống sau. Các lệnh Dockerfile cơ bản: **FROM** (image nền), **RUN** (chạy lệnh lúc build), **COPY** (chép file vào), **ENV** (biến môi trường), **EXPOSE** (khai báo cổng), **USER** (user chạy ứng dụng), **CMD/ENTRYPOINT** (lệnh chạy khi container khởi động). Ví dụ một Dockerfile đóng gói trang web tĩnh:

```
FROM nginx:1.27-alpine                # image nền: nginx bản nhỏ gọn
COPY site/ /usr/share/nginx/html/     # chép trang web vào image
EXPOSE 80
# build:  docker build -t mysite:1.0 .
```

Nguyên tắc tốt: chọn image nền nhỏ, gắn phiên bản cụ thể (nginx:1.27) thay vì latest, không chạy ứng dụng bằng root nếu không cần, không đưa mật khẩu vào image. Red Hat có **UBI (Universal Base Image)**, image nền dựa trên RHEL và được phép phân phối tự do.

Registry: kho lưu trữ và phân phối image

Registry là nơi lưu trữ và phân phối image, giống "kho ứng dụng" cho container. Ví dụ: Docker Hub (công khai), registry.redhat.io và Quay.io của Red Hat, Amazon ECR trên AWS, hoặc registry nội bộ của doanh nghiệp. Tên đầy đủ của image có dạng **registry/namespace/tên:tag**, ví dụ docker.io/library/nginx:1.27. Không ghi registry thì Docker mặc định dùng Docker Hub. Ngoài tag, mỗi image còn có **digest** (mã băm sha256) cố định, đảm bảo kéo về đúng bản đã kiểm duyệt, vì tag có thể bị gắn lại cho bản khác. Quy trình thường gặp: build image, gắn tag theo registry (docker tag), đẩy lên (docker push), rồi máy khác kéo về (docker pull) để chạy.

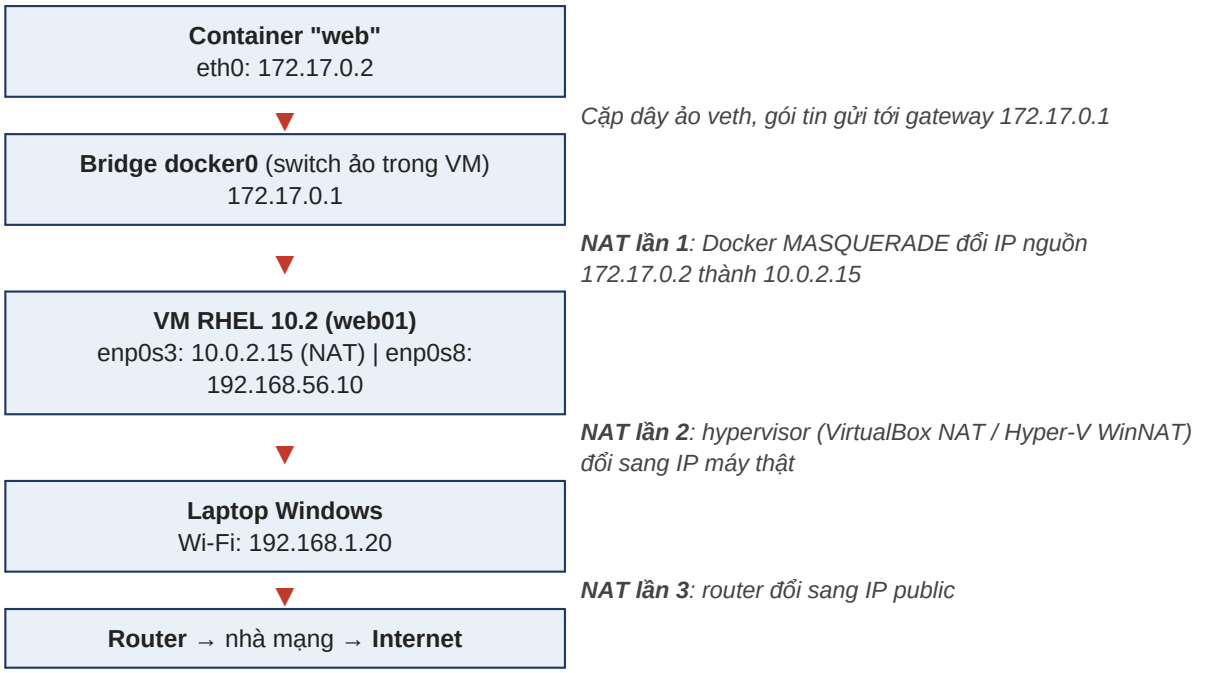
Volume: giữ dữ liệu khi container bị xóa

Container được thiết kế để có thể xóa đi, tạo lại bất cứ lúc nào, và lớp ghi của container mất theo khi bị xóa. Vì vậy dữ liệu cần giữ lâu dài (database, file người dùng tải lên) phải đặt ra ngoài container. Có ba cách. **Named volume** do Docker quản lý (nằm trong /var/lib/docker/volumes/), phù hợp cho database. **Bind mount** gắn một thư mục có sẵn của máy chủ vào container, ví dụ /data/web ở phần 3. **tmpfs** lưu tạm trên RAM, mất khi container dừng. Trên RHEL, SELinux (phần 5) cũng áp dụng cho container: khi bind mount cần thêm hậu tố **:Z** (nhãn riêng cho một container) hoặc **:z** (dùng chung nhiều container), nếu không container sẽ bị chặn truy cập.

Network: container kết nối với nhau, với VM và ra Internet

Docker có nhiều kiểu mạng (network driver). **bridge** là mặc định, các container trên cùng máy nối vào một switch ảo. **host**: container dùng thẳng mạng của máy chủ, không cô lập. **none**: không có mạng. **overlay**: nối container trên nhiều máy. **macvlan**: container có địa chỉ MAC riêng, như một máy thật trong LAN.

Với kiểu bridge, khi tạo container, Docker làm ba việc. Thứ nhất, tạo cho container một **network namespace** riêng, có card eth0, IP (ví dụ 172.17.0.2) và bảng route riêng. Thứ hai, tạo một **cặp veth**, giống sợi dây mạng ảo có hai đầu: một đầu thành eth0 trong container, đầu kia cắm vào bridge **docker0** trên máy chủ. Thứ ba, gán cho docker0 địa chỉ 172.17.0.1 làm **gateway** của container. Như vậy docker0 là một switch ảo bên trong VM, và chính VM đóng vai trò router cho các container. Sơ đồ dưới đây minh họa đường đi của gói tin khi một container chạy trong VM RHEL (trên VirtualBox hoặc Hyper-V, card mạng chế độ NAT) truy cập Internet:



Như vậy gói tin đi qua **4 chặng và 3 lần NAT** để ra Internet, thêm một lần nữa nếu nhà mạng dùng CGNAT. Ở chặng 1, IP đích không cùng mạng 172.17.0.0/16 nên container gửi cho gateway docker0. Ở chặng 2, kernel của VM định tuyến gói tin theo default route (Docker tự bật tính năng chuyển tiếp ip_forward), và rule **MASQUERADE** đổi IP nguồn thành IP của VM. Từ đây, bên ngoài chỉ thấy VM, không thấy container. Ở chặng 3 và 4, hypervisor rồi đến router tiếp tục NAT. Gói tin trả lời đi ngược lại đúng đường đó, vì mỗi lớp NAT có bảng theo dõi kết nối (**conntrack**) để biết phải trả gói tin về cho ai.

Chiều ngược lại, mạng 172.17.x.x chỉ tồn tại bên trong VM, nên máy khác (VM db01, laptop) **không truy cập thẳng được IP container**, mà phải qua **publish port**. Tùy chọn -p 8080:80 tạo một rule **DNAT**: gói tin đến IP của VM ở cổng 8080 được chuyển vào cổng 80 của container. Khi container chủ động kết nối tới VM khác, ví dụ db01 (192.168.56.11), gói tin vẫn đi qua MASQUERADE, nên db01 thấy kết nối đến từ IP của VM chứa container. Các container trong cùng một **user-defined network** nói chuyện trực tiếp qua bridge, không NAT, và gọi nhau bằng tên nhờ DNS nội bộ của Docker (127.0.0.11). Bridge mặc định docker0 không có tính năng phân giải tên này. Ví dụ ứng dụng web và database trên cùng một mạng, database dùng volume để giữ dữ liệu:

```
docker network create appnet
docker run -d --name db  --network appnet -v pgdata:/var/lib/postgresql/data postgres:16
docker run -d --name web --network appnet -p 8080:80 mysite:1.0   # web gọi db bằng tên "db"
```

Lưu ý: Docker tự thêm các rule NAT và chuyển tiếp vào firewall của kernel. Khi máy có firewalld, Docker còn tạo một zone riêng tên **docker**. Vì vậy cổng đã publish có thể truy cập được từ bên ngoài dù chưa khai báo trong firewalld như ở phần 5. Chỉ nên publish những cổng thật sự cần, hoặc giới hạn IP lắng nghe, ví dụ -p 127.0.0.1:8080:80.

Resource limit: giới hạn tài nguyên cho container

Mặc định container được dùng tài nguyên không giới hạn. Một container lỗi (rò rỉ bộ nhớ, vòng lặp vô hạn) có thể chiếm hết RAM, CPU và ảnh hưởng mọi container khác cũng như chính máy chủ. Docker dùng **cgroups v2**, cùng cơ chế systemd dùng ở phần 6, để giới hạn. **--memory** đặt mức RAM tối đa; vượt mức này, kernel sẽ kill tiến trình trong container (trạng thái OOMKilled). **--cpus** giới hạn số CPU được dùng, ví dụ 0.5 là nửa CPU. **--pids-limit** giới hạn số tiến trình. Kết hợp với **restart policy** (tương tự Restart= của systemd), container có thể tự khởi động lại khi gặp lỗi:

```
docker run -d --name web --memory 256m --cpus 0.5 --restart on-failure mysite:1.0
docker stats --no-stream        # CPU, RAM thực tế của từng container
```

Các khái niệm trong phần này (image, registry, volume, network, resource limit) cũng chính là nền tảng của Kubernetes, nội dung em dự định học tiếp ở phần định hướng.

Tổng kết: các mảng kiến thức kết nối với nhau như thế nào

Điều lớn nhất em nhận ra sau giai đoạn này là bảy mảng kiến thức không phải bảy môn học riêng lẻ. Chúng là các lớp chồng lên nhau của cùng một hệ thống: lớp dưới là tiền đề cho lớp trên, và một khái niệm thường xuất hiện lại ở nhiều lớp dưới những tên gọi khác nhau.

Đi từ dưới lên: **hypervisor** cung cấp phần cứng ảo cho máy chủ RHEL. Cấu hình hypervisor ảnh hưởng trực tiếp đến hệ điều hành, ví dụ RHEL 10 không khởi động được nếu VM không được cấp tập lệnh x86-64-v3. Trên nền đó, **network** giúp máy giao tiếp với bên ngoài, **storage** cung cấp chỗ lưu dữ liệu, **user và permission** quyết định ai được truy cập dữ liệu. **systemd** chạy dịch vụ, còn **firewalld và SELinux** bảo vệ dịch vụ đó. **Quản lý tài nguyên** giúp theo dõi toàn bộ hệ thống khi

đang chạy. Cuối cùng, **container** không phải công nghệ tách biệt mà được xây từ chính các tính năng của kernel Linux.

Có ba khái niệm xuất hiện xuyên suốt. **cgroups**: systemd dùng để giới hạn CPU, RAM cho dịch vụ, Docker dùng để giới hạn tài nguyên cho container. **Routing và NAT**: cùng một nguyên lý áp dụng cho VM, cho container (bridge docker0, MASQUERADE, publish port) và cả trên cloud. **Phân quyền nhiều lớp**: quyền file, sudo, SELinux, và với container là nhóm docker có quyền gần như root. Hiểu một khái niệm ở lớp dưới thì học lớp trên nhanh hơn nhiều.

Từ đó em rút ra bốn nguyên tắc tư duy khi làm hạ tầng:

Thứ nhất, nhìn hệ thống theo lớp. Một sự cố đơn giản như "không vào được web" có thể nằm ở mạng, firewall, quyền file, SELinux hoặc dịch vụ đã dừng.

Thứ hai, kiểm tra lần lượt từ dưới lên: IP, route, gateway, DNS, cổng dịch vụ, rồi đến ứng dụng, thay vì đoán mò.

Thứ ba, đọc log trước khi sửa. journalctl, systemctl status và log AVC của SELinux thường đã chỉ ra nguyên nhân.

Thứ tư, luôn có đường lui. Snapshot hoặc backup trước mọi thay đổi lớn, và kiểm tra cấu hình (như mount -a với fstab) trước khi khởi động lại.

Em cũng tự đánh giá thẳng thắn: em đã nắm được phần lý thuyết nền tảng và hiểu "vì sao" phía sau các khái niệm, nhưng **kinh nghiệm thực hành còn rất ít**. Các lệnh trong báo cáo em mới hiểu ở mức đọc và giải thích được, chưa thao tác thành thạo, chưa tự xử lý sự cố trong môi trường thật. Vì vậy ưu tiên số một của giai đoạn tới là biến lý thuyết thành kỹ năng.

Định hướng sắp tới

Lộ trình của em đi theo một logic nối tiếp: trước hết là thực hành để làm chắc nền tảng hiện tại. Sau đó học Kubernetes, bước tự nhiên tiếp theo sau container. Tiếp theo là đưa kiến thức lên cloud với chứng chỉ AWS. Song song là đào sâu Linux doanh nghiệp theo chuẩn Red Hat.

Giai đoạn 1: Thực hành và củng cố nền tảng

Em sẽ chuyển từ "hiểu" sang "làm được" bằng cách tự thực hành lại toàn bộ nội dung trong báo cáo trên RHEL 10.2. Cụ thể: tạo VM trên VirtualBox hoặc Hyper-V, thử snapshot, backup và clone. Cấu hình hai VM kết nối với nhau bằng nmcli. Tạo LVM và mount ổ dữ liệu. Phân quyền cho một nhóm làm việc chung. Chạy dịch vụ web với systemd, firewalld và SELinux. Theo dõi tài nguyên khi giả lập tải. Cuối cùng là chạy ứng dụng web và database bằng Docker, kiểm chứng đường đi mạng như sơ đồ ở phần 7. Em sẽ ghi chép lại các lỗi gặp phải cùng cách xử lý, và tập viết script Bash đơn giản để tự động hóa các việc lặp lại.

Giai đoạn 2: Kubernetes (K8s)

Docker giải quyết bài toán chạy container trên một máy. Kubernetes giải quyết bài toán điều phối hàng trăm container trên nhiều máy: tự khởi động lại container khi lỗi, tự mở rộng khi tải tăng, cập nhật phiên bản không gián đoạn. Hầu hết khái niệm của K8s đều nối với những gì em vừa học. **Service** tương ứng với network và DNS. **PersistentVolume** tương ứng với volume và storage. **Requests và limits** chính là cgroups. **kubelet** trên mỗi node là một dịch vụ systemd. Em dự định dựng một cụm K8s nhỏ bằng kubeadm trên ba VM (một control plane, hai worker) để vừa học K8s, vừa thực hành luôn kỹ năng tạo VM và cấu hình mạng.

Giai đoạn 3: Chứng chỉ AWS Solutions Architect – Associate (SAA-C03)

Kỳ thi SAA-C03 đánh giá khả năng thiết kế giải pháp trên AWS theo Well-Architected Framework, gồm bốn nhóm: bảo mật (30%), khả năng phục hồi (26%), hiệu năng (24%) và tối ưu chi phí (20%). Em chọn chứng chỉ này vì kiến thức nền ánh xạ gần như trực tiếp sang các dịch vụ AWS. **EC2** là VM. **VPC, subnet, route table** là network và routing. **NAT Gateway, Internet Gateway** là các lớp NAT và gateway trong sơ đồ phần 7. **Security Group** là firewall. **EBS, EFS, S3** là storage. **IAM** là user, group và least privilege. **EBS snapshot, AWS Backup** là snapshot và backup. **ECR, ECS, EKS** là registry, container và Kubernetes. **CloudWatch** là theo dõi tài nguyên. Nhờ vậy em không phải học AWS từ con số không, mà học cách các khái niệm quen thuộc vận hành trên cloud. Em sẽ kết hợp học lý thuyết với làm lab trên AWS Free Tier, có đặt cảnh báo chi phí.

Giai đoạn 4: Red Hat (học song song)

Song song với AWS, em sẽ học theo lộ trình chứng chỉ **RHCSA (EX200)**. Đây là kỳ thi thực hành hoàn toàn trên hệ thống RHEL thật, hiện dựa trên RHEL 10, và các bài làm phải vẫn hoạt động đúng sau khi khởi động lại. Vì là thi thực hành nên lộ trình này cũng buộc em phải luyện tay thật nhiều, bổ sung đúng điểm yếu hiện tại. Nội dung thi trùng với nền tảng trong báo cáo: storage và LVM, user và permission, systemd, firewalld, mạng và SELinux. Ngoài ra em sẽ học thêm Podman, công cụ container mặc định trên RHEL. Sau RHCSA, hướng tiếp theo là RHCE (EX294) về tự động hóa bằng Ansible.

Em rất mong nhận được góp ý của anh/chị về lộ trình này, đặc biệt là những kỹ năng mà team đang cần, để em ưu tiên thứ tự học cho phù hợp với công việc thực tế. Em cảm ơn anh/chị đã dành thời gian lắng nghe.